

END-TO-END MLOPS ARCHITECTURE

TRANSACTION FRAUD DETECTION

Documento de Planificación del Proyecto

Versión 1.0 | Abril 2025

9 Fases	63 Tareas	~30 días	12 Servicios
de desarrollo	con subtareas	de esfuerzo total	en el stack

1. Descripción del Proyecto

Este proyecto combina dos iniciativas MLOps de alto impacto en una arquitectura unificada end-to-end:

Fraud Detection en Tiempo Real	MLOps Platform con Drift Detection
Detecta transacciones fraudulentas mientras ocurren, replicando la arquitectura de sistemas como Mercado Pago o Visa. Cada transacción se evalúa en menos de 100ms mediante un pipeline en vivo con feature engineering sobre el stream de Kafka y un modelo XGBoost.	Un modelo en producción que se monitorea solo y avisa cuando empieza a fallar. Tracking con MLflow, reentrenamiento orquestado con Airflow, y detección automática de drift con Evidently AI. PostgreSQL como cerebro del sistema con triggers y stored procedures.

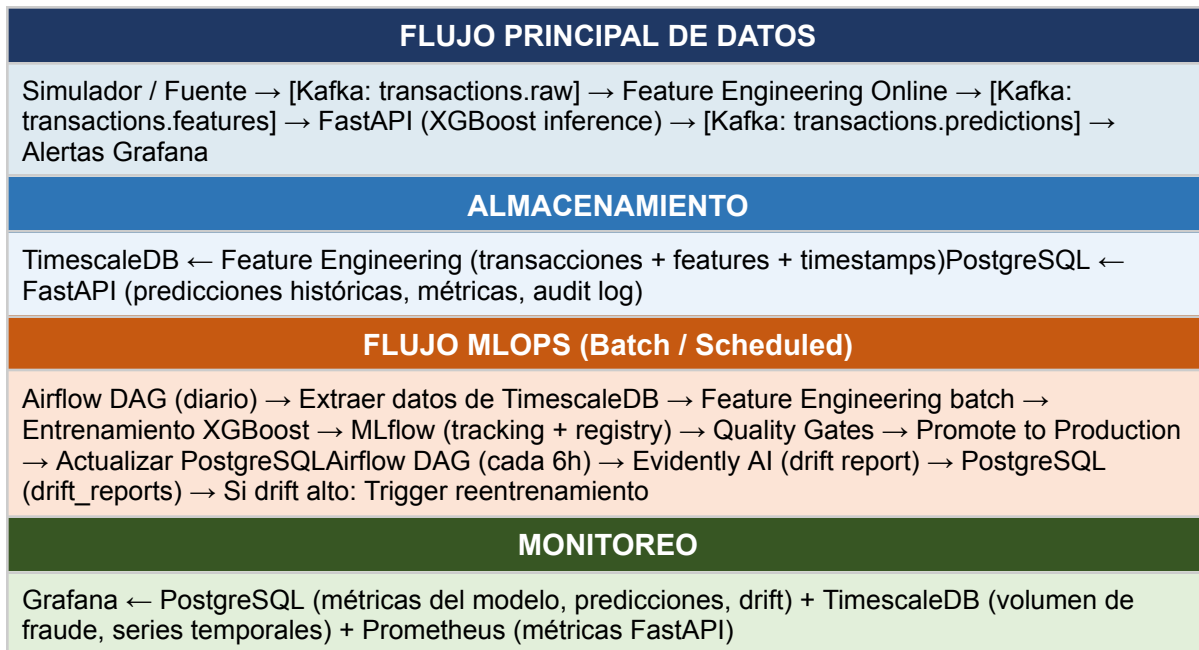
El resultado es una plataforma MLOps completa que demuestra comprensión profunda de producción real: streaming de datos, feature engineering online, serving de baja latencia, reentrenamiento automático y monitoreo continuo del modelo.

2. Stack Tecnológico

Capa	Herramienta	Rol en el sistema
Ingesta & Streaming	Apache Kafka	Stream de transacciones bancarias en tiempo real
Ingesta & Streaming	Kafka Connect	Conectores source/sink para integración con bases de datos
Feature Engineering	Python (custom)	Transformaciones sobre el stream de Kafka (ventanas deslizantes, agregados por usuario, etc.)
Modelo ML	XGBoost	Clasificador binario de fraude con features en tiempo real
Experimentos ML	MLflow	Tracking de experimentos, versiones de modelo, model registry
Serving / API	FastAPI	Endpoint de inferencia con latencia < 100ms
Orquestación	Apache Airflow	DAGs para reentrenamiento automático y pipelines de datos
Drift Detection	Evidently AI	Detección automática de data drift y model drift
DB Series Temporales	TimescaleDB	Almacenamiento de transacciones con timestamps, queries temporales, particionado automático
DB Relacional / Cerebro	PostgreSQL	Predicciones históricas, métricas por versión, triggers de alertas, stored procedures, auditoría
Monitoreo / Alertas	Grafana	Dashboards de fraude, métricas del modelo y salud del sistema

Containerización	Docker + Compose	Empaquetado de todos los servicios y orquestación local
CI/CD	GitHub Actions	Pipeline de integración y despliegue continuo

3. Diagrama de Flujo de Datos



4. Estructura de Carpetas del Proyecto

La siguiente estructura refleja la separación de responsabilidades de cada componente del sistema:

Ruta	Descripción
fraud-mlops/	Raíz del proyecto
— docker/	Dockerfiles por servicio
— kafka/	Imagen customizada de Kafka
— airflow/	Imagen base + dependencias
— fastapi/	Imagen del servidor de inferencia
— grafana/	Provisioning de dashboards
— docker-compose.yml	Orquestación completa del stack
— docker-compose.override.yml	Overrides para desarrollo local
— .env.example	Variables de entorno template
— ingestion/	Capa de ingesta y streaming
— producer/	Simulador / productor de transacciones
— consumer/	Consumidor Kafka + feature engineering
— schemas/	Schemas Avro/JSON de mensajes Kafka
— feature_engineering/	Pipeline de features offline y online
— online/	Features en tiempo real sobre el stream
— offline/	Features batch para entrenamiento
— model/	Desarrollo y entrenamiento del modelo
— train.py	Script principal de entrenamiento
— evaluate.py	Evaluación y métricas del modelo
— predict.py	Lógica de predicción
— features.py	Definición de features
— notebooks/	EDA y experimentos exploratorios
— serving/	API de inferencia FastAPI
— app/	Módulos de la aplicación FastAPI
— main.py	Entry point FastAPI
— routes/	Endpoints REST
— schemas/	Pydantic models
— services/	Lógica de negocio / inferencia
— tests/	Tests de la API
— mlops/	Infraestructura MLOps

— airflow/	DAGs de Airflow
— dags/	Definición de DAGs
— plugins/	Plugins y operators custom
— mlflow/	Configuración MLflow
— evidently/	Reports y configuración de drift
— database/	Schemas, migraciones y seeds
— timescaledb/	Scripts SQL TimescaleDB
— migrations/	Migraciones versionadas
— seeds/	Datos iniciales / de prueba
— postgresql/	Scripts SQL PostgreSQL
— migrations/	Migraciones versionadas
— stored_procedures/	Stored procedures y funciones
— triggers/	Triggers de alertas
— monitoring/	Configuración de monitoreo
— grafana/	Dashboards y datasources
— dashboards/	JSON de dashboards
— provisioning/	Auto-provisioning de Grafana
— alerts/	Reglas de alertas
— tests/	Tests globales del proyecto
— unit/	Tests unitarios
— integration/	Tests de integración
— load/	Tests de carga (Locust/k6)
— .github/workflows/	Pipelines CI/CD GitHub Actions
— scripts/	Scripts utilitarios y de setup
— docs/	Documentación técnica
— architecture.md	Diagrama y descripción de arquitectura
— runbooks/	Procedimientos operativos
— api/	Documentación OpenAPI
— requirements/	Dependencias por componente
— base.txt	Dependencias comunes
— model.txt	Dependencias de modelo
— serving.txt	Dependencias de serving

5. Planificación de Fases y Tareas

A continuación se detallan todas las fases, tareas y subtareas del proyecto organizadas por prioridad y esfuerzo estimado. Leyenda: Alta = crítico para el funcionamiento, Media = importante pero no bloqueante, Baja = mejora o nice-to-have. El esfuerzo está expresado en días/persona.

FASE 1 — Setup e Infraestructura Base

Establecer el entorno de desarrollo, estructura del repositorio, configuración de Docker y la base de todos los servicios del stack. Esta fase es el cimiento del proyecto.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
1 · 1	Inicializar repositorio y estructura de carpetas	Crear repo Git, estructura de directorios según arquitectura, .gitignore, README base, pre-commit hooks.	Alta	1d
1 · 2	Dockerizar todos los servicios	Escribir Dockerfile para cada servicio, docker-compose.yml completo, redes internas, volúmenes persistentes.	Alta	3d
1 · 3	Configurar variables de entorno y secrets	Definir todas las variables de conexión (DB, Kafka, MLflow), gestionar secrets de forma segura.	Alta	0.5d

Detalle de Subtareas

Tarea 1.1: Inicializar repositorio y estructura de carpetas

Nº	Subtarea	Descripción detallada
1. 1. 1	Crear repositorio Git	Inicializar repo en GitHub/GitLab con rama main y develop. Configurar branch protection rules.
1. 1. 2	Crear estructura de carpetas	Generar todos los directorios según la arquitectura definida (ver sección de estructura de carpetas).
1. 1. 3	Configurar .gitignore	Incluir patrones para Python (__pycache__, .env, *.pyc), modelos ML (.pkl, .joblib), datos (data/, *.csv), y logs.
1. 1. 4	Configurar pre-commit hooks	Instalar pre-commit con hooks: black (formatting), flake8 (linting), isort (imports), bandit (security).
1. 1. 5	Crear README.md base	Documentar descripción del proyecto, stack tecnológico, instrucciones de setup, y diagrama de arquitectura simplificado.
1. 1. 6	Crear .env.example	Definir todas las variables de entorno necesarias con valores placeholder y documentación inline.

Tarea 1.2: Dockerizar todos los servicios

Nº	Subtarea	Descripción detallada
----	----------	-----------------------

1. 2. 1	Dockerfile — FastAPI	Imagen Python 3.11-slim, instalar dependencias de serving, copiar código, healthcheck, user no-root.
1. 2. 2	Dockerfile — Airflow	Extender imagen oficial apache/airflow:2.8, agregar dependencias custom (mlflow, evidently, psycpg2).
1. 2. 3	Dockerfile — Kafka Producer	Imagen Python para el simulador de transacciones. Configurar variables de entorno para Kafka brokers.
1. 2. 4	Dockerfile — Grafana	Extender imagen oficial con provisioning automático de datasources y dashboards.
1. 2. 5	docker-compose.yml principal	Definir todos los servicios: Kafka, Zookeeper, TimescaleDB, PostgreSQL, MLflow, Airflow (webserver + scheduler), FastAPI, Grafana, Evidently. Configurar redes internas (mlops-net) y volúmenes persistentes.
1. 2. 6	docker-compose.override.yml	Overrides para desarrollo: montar código como volumen (hot reload), exponer puertos extra, reducir recursos.
1. 2. 7	Script de setup inicial	Script scripts/setup.sh que cree .env desde .env.example, levante el stack, y ejecute migraciones de DB.

Tarea 1.3: Configurar variables de entorno y secrets

N°	Subtarea	Descripción detallada
1. 3. 1	Definir variables por servicio	Variables para PostgreSQL (host, port, user, password, db), TimescaleDB, Kafka brokers, MLflow tracking URI, Airflow Fernet key.
1. 3. 2	Implementar config loader	Módulo Python (config.py) usando pydantic-settings para cargar y validar variables de entorno en todos los servicios.

FASE 2 — Bases de Datos

Diseñar e implementar los dos esquemas de base de datos: TimescaleDB para series temporales de transacciones y PostgreSQL como "cerebro" del sistema con stored procedures, triggers y auditoría.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
2.1	TimescaleDB — Diseño de esquema	Crear hipertablas de transacciones, índices temporales, particionado automático, vistas materializadas.	Alta	1.5d
2.2	PostgreSQL — Diseño del sistema cerebro	Tablas de predicciones, métricas por versión de modelo, triggers de alertas, stored procedures, auditoría.	Alta	2d
2.3	Migraciones y seeds de prueba	Sistema de migraciones versionado, datos de prueba para desarrollo.	Media	1d

Detalle de Subtareas

Tarea 2.1: TimescaleDB — Diseño de esquema

Nº	Subtarea	Descripción detallada
2.1.1	Crear tabla transactions (hypertable)	Campos: transaction_id, user_id, merchant_id, amount, country, timestamp, is_fraud (nullable), model_score, latency_ms. Ejecutar create_hypertable con chunk_time_interval de 1 día.
2.1.2	Crear índices optimizados	Índice compuesto (user_id, timestamp) para queries de historial de usuario. Índice en (timestamp) para queries temporales. Índice parcial en is_fraud = true para dashboards.
2.1.3	Crear vistas continuas (cagg)	Vista materializada de volumen de fraude por hora. Vista de monto total por merchant por día. Configurar refresh policy cada 5 minutos.
2.1.4	Crear políticas de retención	Política para comprimir datos mayores a 7 días. Política de drop para datos mayores a 2 años.
2.1.5	Migración inicial	Crear archivo database/timescaledb/migrations/001_initial_schema.sql. Verificar idempotencia con IF NOT EXISTS.

Tarea 2.2: PostgreSQL — Diseño del sistema cerebro

Nº	Subtarea	Descripción detallada
----	----------	-----------------------

2. 2. 1	Crear tabla model_versions	Campos: id, model_name, version, mlflow_run_id, created_at, is_active, f1_score, precision, recall, auc_roc, training_data_from, training_data_to.
2. 2. 2	Crear tabla predictions_history	Campos: id, transaction_id, model_version_id, prediction_score, prediction_label, actual_label (nullable), timestamp, latency_ms.
2. 2. 3	Crear tabla drift_reports	Campos: id, report_date, model_version_id, drift_score, feature_drifts (jsonb), alert_triggered, remediation_action.
2. 2. 4	Crear tabla alert_log	Campos: id, alert_type, severity, message, triggered_at, acknowledged_at, acknowledged_by.
2. 2. 5	Crear stored procedure: activate_model_version	Procedure que desactiva todas las versiones anteriores y activa la nueva. Registra el cambio en audit_log.
2. 2. 6	Crear stored procedure: calculate_model_metrics	Calcula precision, recall, F1 usando predictions_history para un rango de fechas. Actualiza model_versions.
2. 2. 7	Crear trigger: alert_on_high_fraud_rate	Trigger que monitorea la tasa de fraude en ventana de 15 minutos. Si supera el umbral configurable, inserta en alert_log y puede llamar a pg_notify.
2. 2. 8	Crear tabla audit_log	Auditoría general: tabla, operación (INSERT/UPDATE/DELETE), usuario, timestamp, valores old/new en jsonb.
2. 2. 9	Crear trigger de auditoría genérico	Función PL/pgSQL audit_trigger() que se aplica a model_versions, predictions_history para registrar todos los cambios.

Tarea 2.3: Migraciones y seeds de prueba

N°	Subtarea	Descripción detallada
2. 3. 1	Configurar Flyway o Alembic	Elegir herramienta de migraciones. Si Python: Alembic con alembic.ini para cada DB. Si SQL puro: scripts numerados + script runner.
2. 3. 2	Crear seed de transacciones de prueba	Script Python que genere 10,000 transacciones sintéticas con distribución realista (1-3% de fraude) e inserte en TimescaleDB.
2. 3. 3	Crear seed de model_versions de prueba	Insertar versión de modelo v0.1 como baseline con métricas placeholder para poder iniciar el sistema.

FASE 3 — Ingesta y Streaming con Kafka

Implementar el pipeline de streaming de transacciones en tiempo real con Kafka, incluyendo el simulador de datos, schemas de mensajes y consumidores con feature engineering online.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
3.1.1	Configurar cluster Kafka	Levantar Kafka + Zookeeper (o KRaft), crear topics, configurar retención, replicación.	Alta	1d
3.1.2	Desarrollar simulador de transacciones (Producer)	Generar transacciones sintéticas realistas con distribución estadística, inyectar patrones de fraude.	Alta	2d
3.1.3	Feature Engineering en tiempo real (Consumer)	Consumir stream de Kafka, calcular features en ventanas deslizantes, publicar en topic de features.	Alta	3d

Detalle de Subtareas

Tarea 3.1: Configurar cluster Kafka

Nº	Subtarea	Descripción detallada
3.1.1	Configurar Kafka en Docker Compose	Usar imagen confluentinc/cp-kafka. Configurar broker con KAFKA_ADVERTISED_LISTENERS para acceso interno y externo.
3.1.2	Crear topics	Topic: transactions.raw (mensajes de entrada), transactions.features (features calculadas), transactions.predictions (resultados), transactions.fraud.alerts (alertas). Configurar particiones, retención de 7 días.
3.1.3	Configurar Schema Registry	Levantar Confluent Schema Registry. Definir schemas Avro para cada topic. Garantizar compatibilidad backward.
3.1.4	Configurar Kafka UI	Levantar Kafka UI (provectus/kafka-ui) para monitoreo y debugging en desarrollo.

Tarea 3.2: Desarrollar simulador de transacciones (Producer)

Nº	Subtarea	Descripción detallada
3.2.1	Modelo de datos de transacción	Definir dataclass Transaction: transaction_id (UUID), user_id, merchant_id, merchant_category, amount, country, timestamp, device_type, ip_hash.

3. 2. 2	Generador de transacciones legítimas	Distribución log-normal para amounts, usuarios activos con historial de comportamiento, horarios realistas según día/hora.
3. 2. 3	Generador de patrones de fraude	Implementar 4 patrones: monto atípico, país inusual para el usuario, frecuencia alta en poco tiempo, merchant desconocido con amount alto.
3. 2. 4	Kafka Producer con configuración	Usar confluent-kafka-python. Configurar serialización Avro, retries, acks=all para durabilidad. Tasa configurable de TPS.
3. 2. 5	Modo de testing	Flag --replay para reproducir transacciones históricas desde CSV. Flag --scenario para inyectar escenarios específicos de fraude.

Tarea 3.3: Feature Engineering en tiempo real (Consumer)

Nº	Subtarea	Descripción detallada
3. 3. 1	Implementar consumer base	Consumer group fraud-feature-engineering. Deserializar Avro. Manejar errores y dead-letter queue.
3. 3. 2	Features de ventana temporal (sliding window)	Para cada user_id: conteo de transacciones en últimas 1h/24h/7d, suma de montos en últimas 1h/24h, velocidad (txns/hora), tiempo desde última transacción.
3. 3. 3	Features de comportamiento histórico	Ratio entre monto actual y monto promedio del usuario (últimas 30d), países visitados históricamente, merchants visitados.
3. 3. 4	Implementar feature store en Redis o TimescaleDB	Cachear estado de usuario (últimas N transacciones) en Redis para lookup de baja latencia en ventanas temporales.
3. 3. 5	Publicar features procesadas	Serializar TransactionWithFeatures y publicar en topic transactions.features para consumo por el modelo.
3. 3. 6	Guardar features en TimescaleDB	Insertar cada transacción procesada con sus features calculadas y timestamp en la hypertable de TimescaleDB.

FASE 4 — Modelo ML con XGBoost

Desarrollar el pipeline completo de entrenamiento del modelo XGBoost para detección de fraude, desde la exploración de datos hasta el registro de modelos en MLflow.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
4.1.1	Exploración y análisis de datos (EDA)	Notebook de EDA: distribución de clases, correlaciones, distribución de features, análisis temporal.	Alta	2d
4.1.2	Feature Engineering offline (training)	Pipeline de transformación para entrenamiento: encoding, scaling, manejo de desbalance de clases, feature selection.	Alta	2d
4.1.3	Entrenamiento del modelo XGBoost	Pipeline de entrenamiento, hyperparameter tuning, evaluación con métricas de fraude, registro en MLflow.	Alta	2d
4.1.4	Validación y promoción de modelo	Gates de calidad antes de promover a producción, comparación con modelo baseline.	Alta	1d

Detalle de Subtareas

Tarea 4.1: Exploración y análisis de datos (EDA)

N°	Subtarea	Descripción detallada
4.1.1	Notebook EDA base	Cargar datos de TimescaleDB. Análisis de distribución de clases (fraude vs. legítimo). Visualización de distribución de amounts, hours, countries.
4.1.2	Análisis de correlación de features	Matriz de correlación. Importance plot inicial con modelo simple. Identificar features redundantes y candidatas a eliminar.
4.1.3	Análisis temporal	Patrones de fraude por hora del día, día de la semana, mes. Identificar estacionalidad que afecte al modelo.
4.1.4	Documentar hallazgos	Crear docs/eda_findings.md con insights, decisiones de feature engineering y recomendaciones para el modelo.

Tarea 4.2: Feature Engineering offline (training)

N°	Subtarea	Descripción detallada
4.2.1	Implementar TransactionFeaturizer	Clase Python que implementa las mismas features que el pipeline online (para garantizar consistencia train/serve). Usar sklearn Pipeline.

4. 2. 2	Encoding de variables categóricas	Target encoding para merchant_category y country. Ordinal encoding para device_type. Guardar encoders como artefactos en MLflow.
4. 2. 3	Manejo de desbalance de clases	Implementar SMOTE para oversampling de la clase fraude. Alternativamente, configurar scale_pos_weight en XGBoost. Evaluar ambos enfoques.
4. 2. 4	Feature selection	Usar XGBoost feature importance + Boruta para seleccionar features relevantes. Documentar features finales en model/features.py.

Tarea 4.3: Entrenamiento del modelo XGBoost

N°	Subtarea	Descripción detallada
4. 3. 1	Implementar script train.py	Cargar datos desde TimescaleDB, aplicar feature pipeline, split temporal (no aleatorio) train/validation/test, entrenar XGBoost.
4. 3. 2	Integrar MLflow tracking	mlflow.start_run() al inicio. Loggear: parámetros de XGBoost, métricas (F1, precision, recall, AUC-ROC, FPR, FNR), artefactos (feature importances, confusion matrix, ROC curve).
4. 3. 3	Hyperparameter tuning	Usar Optuna o sklearn GridSearchCV. Optimizar: n_estimators, max_depth, learning_rate, min_child_weight, scale_pos_weight. Loggear cada trial en MLflow.
4. 3. 4	Evaluación con métricas de negocio	Calcular costo del fraude no detectado vs. costo de falsos positivos. Optimizar threshold de clasificación según business objective.
4. 3. 5	Registrar modelo en MLflow Model Registry	mlflow.xgboost.log_model() con signature inferida. Registrar como FraudDetectionModel v1. Transicionar a Staging.

Tarea 4.4: Validación y promoción de modelo

N°	Subtarea	Descripción detallada
4. 4. 1	Implementar quality gates	Script evaluate.py que rechaza el modelo si: F1 < 0.85, AUC-ROC < 0.90, latencia de predicción > 50ms en batch de 1000.
4. 4. 2	Comparación con modelo en producción	Comparar challenger vs. champion en métricas clave. Solo promover si mejora significativa (> 2% F1).
4. 4. 3	Promover modelo a Production	Si pasa quality gates: transicionar en MLflow Registry a Production. Registrar en tabla model_versions de PostgreSQL con stored procedure activate_model_version.

FASE 5 — Serving con FastAPI

Implementar el endpoint de inferencia de alta performance con FastAPI, cargando el modelo desde MLflow Registry y garantizando latencia menor a 100ms.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
5.1	Implementar API de inferencia	Endpoints REST para predicción individual y batch, carga de modelo desde MLflow, schemas Pydantic.	Alta	2d
5.2	Optimización de latencia	Garantizar latencia < 100ms bajo carga, profiling, caché de feature lookup.	Alta	1d
5.3	Integración con Kafka para inferencia streaming	Consumer que consume del topic de features y llama al modelo, publicando resultados.	Alta	1.5d

Detalle de Subtareas

Tarea 5.1: Implementar API de inferencia

Nº	Subtarea	Descripción detallada
5.1.1	Configurar estructura FastAPI	Crear serving/app/main.py con lifespan para carga de modelo al startup. Configurar CORS, middlewares de logging y métricas.
5.1.2	Implementar model loader	Servicio que carga el modelo Production desde MLflow Registry al iniciar. Soportar reload sin downtime con versión en memoria.
5.1.3	Endpoint POST /predict	Recibe TransactionRequest (Pydantic), aplica feature pipeline, infiere con XGBoost, retorna PredictionResponse: score, label, latency_ms, model_version.
5.1.4	Endpoint POST /predict/batch	Recibe lista de transacciones, procesa en batch para mayor throughput. Límite configurable de batch size.
5.1.5	Endpoint GET /health y GET /model/info	Health check con estado del modelo cargado. Model info con versión activa, métricas de entrenamiento, fecha de deploy.
5.1.6	Guardar predicciones en PostgreSQL	Async background task que inserta cada predicción en predictions_history con transaction_id, score, model_version, latency_ms.

Tarea 5.2: Optimización de latencia

Nº	Subtarea	Descripción detallada
----	----------	-----------------------

5. 2. 1	Profiling de latencia	Medir latencia end-to-end de /predict: feature computation + model inference + DB write. Identificar bottlenecks.
5. 2. 2	Optimizar feature computation	Caché de features de usuario en Redis. Vectorizar cálculos con numpy. Precargar encoders al startup.
5. 2. 3	Async DB writes	Usar asyncpg para escrituras asíncronas en PostgreSQL. No bloquear la respuesta esperando DB write.
5. 2. 4	Configurar workers y concurrencia	Configurar Uvicorn con múltiples workers. Implementar connection pooling para PostgreSQL.

Tarea 5.3: Integración con Kafka para inferencia streaming

N°	Subtarea	Descripción detallada
5. 3. 1	Implementar inference consumer	Consumer del topic transactions.features. Para cada mensaje: llamar a FastAPI /predict o directamente al modelo en memoria. Publicar resultado en transactions.predictions.
5. 3. 2	Publicar alertas de fraude	Si prediction_label == FRAUD y score > threshold: publicar en transactions.fraud.alerts con contexto completo.
5. 3. 3	Manejar backpressure	Configurar consumer con manejo de backpressure. Si API saturada, usar circuit breaker para degradar gracefully.

FASE 6 — MLOps: Airflow + MLflow + Evidently

Implementar la infraestructura MLOps completa: tracking de experimentos con MLflow, orquestación de reentrenamiento automático con Airflow y detección de drift con Evidently AI.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
6.1	Configurar MLflow	MLflow tracking server, artifact store, model registry, configuración de retención.	Alta	1d
6.2	Desarrollar DAGs de Airflow	DAG de reentrenamiento automático, DAG de validación de modelo, DAG de reporte de métricas.	Alta	3d
6.3	Implementar detección de drift con Evidently AI	Reports automáticos de data drift y model drift, integración con Airflow y PostgreSQL.	Alta	2d

Detalle de Subtareas

Tarea 6.1: Configurar MLflow

Nº	Subtarea	Descripción detallada
6.1.1	Levantar MLflow Tracking Server	Configurar con backend store en PostgreSQL (--backend-store-uri) y artifact store en volumen local o S3-compatible (MinIO). Exponer en puerto 5000.
6.1.2	Configurar experimentos	Crear experimento fraud-detection-v1. Configurar tags de experimento: project, team, data_version.
6.1.3	Definir modelo en registry	Registrar modelo FraudDetectionModel con stages: None → Staging → Production → Archived.

Tarea 6.2: Desarrollar DAGs de Airflow

Nº	Subtarea	Descripción detallada
6.2.1	Configurar Airflow	Configurar LocalExecutor o CeleryExecutor. Definir conexiones Airflow: postgres_conn, mlflow_conn, timescaledb_conn.
6.2.2	DAG: retrain_fraud_model	Schedule: diario a las 2:00 AM. Tasks: extract_training_data (desde TimescaleDB), feature_engineering, train_xgboost (con MLflow), evaluate_model, quality_gates, register_model.
6.2.3	DAG: validate_and_promote_model	Triggered por retrain_fraud_model al registrar nuevo modelo. Tasks: compare_with_champion, shadow_testing, promote_to_production, update_postgresql, notify_team.

6. 2. 4	DAG: drift_detection_report	Schedule: cada 6 horas. Tasks: fetch_recent_predictions, fetch_training_reference_data, run_evidently_report, evaluate_drift_threshold, trigger_retrain_if_needed.
6. 2. 5	DAG: data_quality_check	Schedule: cada hora. Verificar: no hay gaps en el stream de Kafka, distribución de amounts no anómala, rate de predicciones dentro de rango.
6. 2. 6	Implementar custom operators	MLflowRegisterModelOperator, EvidentlyReportOperator, TimescaleExtractOperator para reusar lógica.

Tarea 6.3: Implementar detección de drift con Evidently AI

N°	Subtarea	Descripción detallada
6. 3. 1	Definir dataset de referencia	Guardar distribución de features del dataset de entrenamiento como referencia. Artefacto en MLflow y copia en PostgreSQL.
6. 3. 2	Implementar reporte de data drift	EvidentlyReport con DataDriftPreset. Comparar features de producción (últimas 24h) contra referencia. Calcular drift score por feature.
6. 3. 3	Implementar reporte de model drift (concept drift)	ClassificationPreset para monitorear precision/recall/F1 en producción con labels reales (cuando disponibles). Target drift detection.
6. 3. 4	Guardar reportes en PostgreSQL	Insertar en drift_reports: fecha, score global, drift por feature (jsonb), si se disparó alerta, acción tomada.
6. 3. 5	Configurar thresholds y acciones	Si drift_score > 0.3 en cualquier feature crítica: insertar en alert_log con severity HIGH, trigger DAG de reentrenamiento.
6. 3. 6	Exportar HTML reports	Guardar reports HTML de Evidently en artefactos de MLflow para auditoría y revisión manual.

FASE 7 — Monitoreo con Grafana

Implementar dashboards completos en Grafana para monitoreo de alertas de fraude, métricas del modelo y salud del sistema, con provisioning automático.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
7.1	Configurar datasources en Grafana	Conectar Grafana a TimescaleDB, PostgreSQL y endpoint de métricas de FastAPI.	Alta	0.5d
7.2	Desarrollar dashboards	Dashboard de alertas de fraude, dashboard de métricas del modelo, dashboard de salud del sistema.	Alta	2.5d

Detalle de Subtareas

Tarea 7.1: Configurar datasources en Grafana

Nº	Subtarea	Descripción detallada
7.1.1	Provisioning automático de datasources	Archivos YAML en monitoring/grafana/provisioning/datasources/. Datasources: timescaledb (PostgreSQL plugin), postgresql, prometheus (para métricas FastAPI).
7.1.2	Configurar Prometheus + métricas FastAPI	Agregar prometheus-fastapi-instrumentator a FastAPI. Exponer /metrics. Configurar Prometheus scrape.

Tarea 7.2: Desarrollar dashboards

Nº	Subtarea	Descripción detallada
7.2.1	Dashboard: Fraud Detection — Alertas en vivo	Panels: Tasa de fraude en tiempo real (gauge), Volumen de transacciones/minuto, Mapa de calor de fraude por hora/día, Alertas recientes (tabla con transaction_id, score, merchant), Top 10 merchants con más fraude.
7.2.2	Dashboard: Model Performance	Panels: F1 score por versión de modelo (serie temporal), Precision vs. Recall (gráfico dual), Distribución de scores del modelo (histograma), Latencia P50/P95/P99 de /predict, Comparación champion vs. challenger.
7.2.3	Dashboard: Data Drift Monitor	Panels: Drift score por feature (heatmap), Evolución temporal del drift score, Alertas de drift (tabla), Estado del modelo (semáforo: OK/WARNING/DRIFT), Próximo reentrenamiento programado.
7.2.4	Dashboard: System Health	Panels: Estado de todos los servicios (Kafka, FastAPI, Airflow, DBs), Kafka consumer lag, DAGs de Airflow en ejecución, Uso de CPU/memoria/disco por servicio.

7. 2. 5	Configurar alertas en Grafana	Alert rules: tasa de fraude > 5% por 10min → Slack/email, Kafka consumer lag > 1000 mensajes, FastAPI P99 > 200ms, DAG falla 2 veces consecutivas.
---------------	--------------------------------------	--

FASE 8 — Testing

Implementar suite completa de tests: unitarios, de integración y de carga para garantizar la calidad y performance del sistema.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
8.1	Tests unitarios	Tests de feature engineering, modelo, API endpoints, DAGs de Airflow.	Alta	2d
8.2	Tests de integración	Tests end-to-end de flujos críticos con servicios reales (testcontainers).	Alta	2d
8.3	Tests de carga y performance	Verificar latencia < 100ms bajo carga, throughput del sistema.	Media	1d

Detalle de Subtareas

Tarea 8.1: Tests unitarios

Nº	Subtarea	Descripción detallada
8.1.1	Tests de feature engineering	Verificar que cada feature se calcula correctamente con datos conocidos. Testear casos edge: usuario nuevo (sin historial), transacción de monto 0, país desconocido.
8.1.2	Tests del modelo	Verificar que el modelo carga correctamente. Testear predicciones con casos conocidos de fraude y no-fraude. Verificar que la signature del modelo no ha cambiado.
8.1.3	Tests de API (pytest + httpx)	Test /predict con transacción válida e inválida. Test /predict/batch. Test /health. Mock de MLflow para evitar dependencia externa.
8.1.4	Tests de DAGs de Airflow	Verificar estructura del DAG (sin ciclos, tareas conectadas). Testear cada operator individualmente con mocks de DB y MLflow.

Tarea 8.2: Tests de integración

Nº	Subtarea	Descripción detallada
8.2.1	Test: flujo completo productor → features → predicción	Usar testcontainers para Kafka, TimescaleDB, PostgreSQL. Enviar transacción, verificar que llega a prediction, se guarda en DB, y aparece en TimescaleDB.
8.2.2	Test: entrenamiento y registro de modelo	Ejecutar pipeline de entrenamiento completo con datos mini. Verificar que se registra en MLflow y en PostgreSQL. Verificar quality gates.

8. 2. 3	Test: drift detection y trigger de reentrenamiento	Inyectar datos con drift conocido. Verificar que Evidently lo detecta, se inserta en drift_reports, y se dispara el DAG de reentrenamiento.
---------------	---	---

Tarea 8.3: Tests de carga y performance

Nº	Subtarea	Descripción detallada
8. 3. 1	Test de carga FastAPI (Locust)	500 usuarios concurrentes enviando a /predict. Verificar P99 < 100ms, 0% error rate. Reportar throughput máximo (TPS).
8. 3. 2	Test de throughput Kafka	Enviar 10,000 transacciones/minuto. Verificar que el consumer mantiene el lag < 100 mensajes. Verificar que TimescaleDB no es bottleneck.
8. 3. 3	Test de queries TimescaleDB	Benchmarks de queries críticas: volumen de fraude por hora últimas 24h, historial de usuario (< 50ms). Verificar que los índices y cagg funcionan.

FASE 9 — CI/CD y Despliegue

Implementar el pipeline de CI/CD completo con GitHub Actions y el proceso de despliegue del stack completo.

#	Tarea	Subtareas / Descripción	Prioridad	Esfuerzo
9.1	Implementar pipeline CI con GitHub Actions	Pipeline de CI: linting, tests, build de imágenes Docker, security scan.	Alta	2d
9.2	Despliegue del stack completo	Proceso de despliegue documentado, scripts de bootstrap, verificación de salud post-deploy.	Alta	2d
9.3	Documentación final del proyecto	README completo, documentación de arquitectura, OpenAPI docs, guías de contribución.	Media	1.5d

Detalle de Subtareas

Tarea 9.1: Implementar pipeline CI con GitHub Actions

Nº	Subtarea	Descripción detallada
9.1.1	Workflow: lint-and-test.yml	Trigger: push a cualquier branch. Jobs: install deps, run flake8, run black --check, run isort --check, run pytest unitarios con coverage > 80%.
9.1.2	Workflow: integration-tests.yml	Trigger: PR a develop y main. Jobs: levantar servicios con docker-compose, ejecutar tests de integración, reportar resultados.
9.1.3	Workflow: build-and-push.yml	Trigger: merge a main. Jobs: build Docker images, tag con SHA y versión semántica, push a container registry (ghcr.io o Docker Hub).
9.1.4	Workflow: security-scan.yml	Ejecutar Trivy en imágenes Docker para detectar vulnerabilidades. Ejecutar bandit en código Python. Fail si vulnerabilidades CRITICAL.

Tarea 9.2: Despliegue del stack completo

Nº	Subtarea	Descripción detallada
9.2.1	Script de bootstrap completo	scripts/deploy.sh: copiar .env, ejecutar docker-compose pull, ejecutar docker-compose up -d, esperar healthchecks, ejecutar migraciones, cargar seed de prueba, verificar todos los servicios.
9.2.2	Script de verificación post-deploy	scripts/smoke_test.sh: verificar que cada servicio responde (Kafka, FastAPI /health, Grafana, MLflow, Airflow). Enviar transacción de prueba y verificar predicción.

9. 2. 3	Configurar orden de startup y healthchecks	Definir depends_on con condition: service_healthy en docker-compose.yml. Garantizar que TimescaleDB y PostgreSQL están listos antes de FastAPI y Airflow.
9. 2. 4	Runbook de operaciones	docs/runbooks/: cómo reiniciar servicios individualmente, cómo promover un modelo manualmente, cómo resolver consumer lag, cómo restaurar desde backup.

Tarea 9.3: Documentación final del proyecto

N°	Subtarea	Descripción detallada
9. 3. 1	README.md completo	Descripción del proyecto, diagrama de arquitectura, quickstart (5 comandos para levantar el stack), descripción de cada componente, links a documentación.
9. 3. 2	Documentación de arquitectura	docs/architecture.md: diagrama de flujo de datos (Mermaid), descripción de cada capa, decisiones de diseño y alternativas consideradas.
9. 3. 3	OpenAPI / Swagger docs	Completar docstrings de FastAPI endpoints. Configurar ReDoc en /redoc. Documentar schemas de request/response con ejemplos.
9. 3. 4	CONTRIBUTING.md	Guía para contribuidores: cómo hacer setup local, convenciones de código, proceso de PR, cómo agregar nuevas features.

6. Resumen de Esfuerzo y Timeline

Estimación orientativa de esfuerzo por fase (asumiendo 1 desarrollador full-time):

Fase	Descripción	Esfuerzo	Timeline	Hito
Fase 1	Setup e Infraestructura Base	4.5 días	Semana 1	Stack levantado
Fase 2	Bases de Datos	4.5 días	Semana 1-2	DBs operativas
Fase 3	Ingesta y Streaming con Kafka	6 días	Semana 2-3	Stream funcionando
Fase 4	Modelo ML con XGBoost	7 días	Semana 3-4	Modelo entrenado
Fase 5	Serving con FastAPI	4.5 días	Semana 5	API en vivo
Fase 6	MLOps: Airflow + MLflow + Evidently	6 días	Semana 5-6	Reentrenamiento automático
Fase 7	Monitoreo con Grafana	3 días	Semana 7	Dashboards activos
Fase 8	Testing	5 días	Semana 7-8	Cobertura de tests
Fase 9	CI/CD y Despliegue	5.5 días	Semana 8	Deploy completo
TOTAL	Proyecto completo end-to-end	~46 días	8-9 semanas	MVP en producción

Nota: Los tiempos pueden variar según la familiaridad del equipo con las herramientas. Se recomienda comenzar por las Fases 1-2 en paralelo con la Fase 4 (EDA y modelo) para optimizar el tiempo total del proyecto.

7. Criterios de Éxito del Proyecto

Criterio	Métrica objetivo	Herramienta de verificación
Latencia de inferencia	P99 < 100ms bajo carga de 500 usuarios concurrentes	Locust + Grafana
Precisión del modelo	F1-score ≥ 0.85 , AUC-ROC ≥ 0.90 en test set	MLflow + evaluate.py
Disponibilidad del stream	0 mensajes perdidos bajo carga de 10,000 TPS	Kafka consumer lag + Grafana
Detección de drift	Evidently detecta drift inyectado artificialmente en < 6h	Airflow DAG + drift_reports
Reentrenamiento automático	Nuevo modelo promovido sin intervención manual	Airflow + MLflow Registry
Cobertura de tests	Cobertura > 80% en código de feature engineering y serving	pytest-cov + GitHub Actions
Queries temporales	Consultas de volumen de fraude por hora en < 50ms	TimescaleDB EXPLAIN ANALYZE
Auditoría completa	Todo cambio de modelo registrado en audit_log con usuario y timestamp	PostgreSQL audit_log